EX.NO.: 10
DATE: 04.08.2025

USB ACTIVITY MONITOR

AIM
To design and implement a USB activity monitoring system that detects unauthorized or rogue USB insertions using real-time device tracking and anomaly detection with Isolation Forest.

ALGORITHM
1. Initialize a local SQLite database to log USB device data.
2. Use WMI (`pywin32`) to detect USB insert events in real time.
3. On insertion, extract device metadata: Device ID, Vendor ID, Product ID, Serial Number.
4. Log the device into the database, tagging it as known or unknown.
5. Train an IsolationForest model using features of known devices.
6. When a new device is inserted, classify it as normal or anomalous using the trained model.
7. Alert the user via console if an anomalous device is detected.

CODE AND OUTPUT
```
import win32com.client
import pythoncom
import sqlite3
import threading
import pandas as pd
from sklearn.ensemble import IsolationForest
from datetime import datetime
import os
import re  # Make sure this is imported at the top
import time

DB_NAME = 'usb_devices.db'

# ------------------ DATABASE SETUP ------------------
def init_db():
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS usb_log (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp TEXT,
            device_id TEXT,
            vendor_id TEXT,
            product_id TEXT,
            serial_number TEXT,
            known INTEGER,
            anomaly_status TEXT
        )
    ''')
    conn.commit()
    conn.close()
```

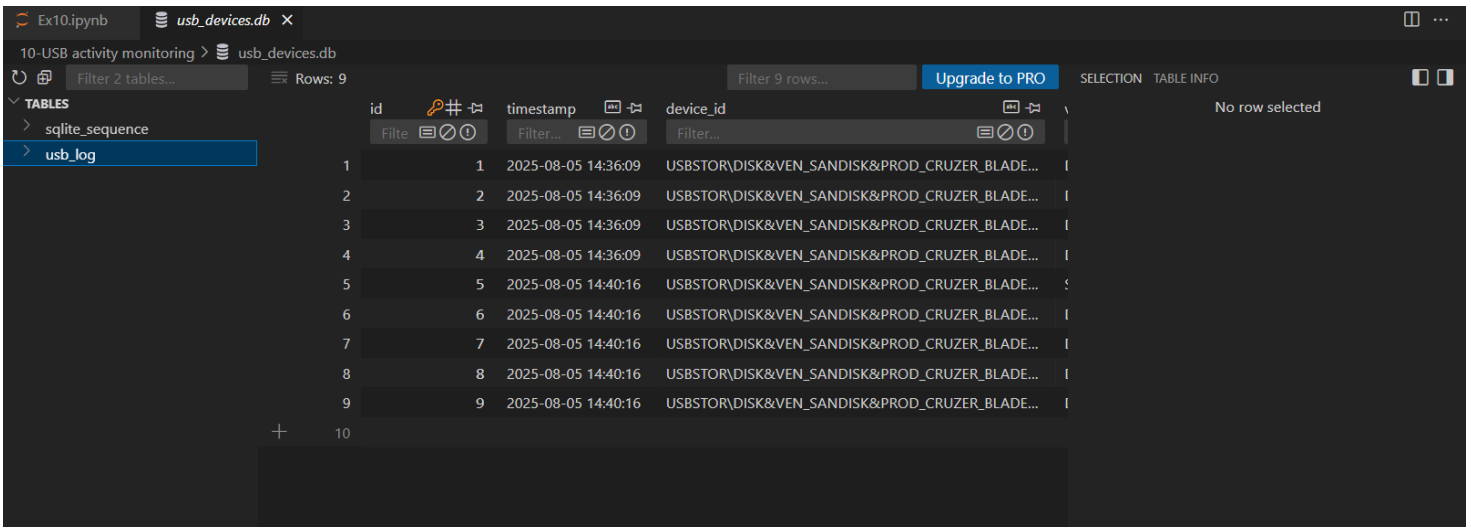
```
# ------------------- USB EVENT HANDLER -------------------
class USBEventHandler:
    def __init__(self):
        self.model = None
        self.columns = None
        self.load_model()

    def load_model(self):
        conn = sqlite3.connect(DB_NAME)
        df = pd.read_sql_query("SELECT vendor_id, product_id, known FROM usb_log WHERE
known=1", conn)
        conn.close()

        if not df.empty:
            X_train = df[["vendor_id", "product_id"]].astype(str)
            X_train_encoded = pd.get_dummies(X_train)
            self.model = IsolationForest(contamination=0.1, random_state=42)
            self.model.fit(X_train_encoded)
            self.columns = X_train_encoded.columns

    def on_usb_inserted(self, device):
        timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        device_id = device.PNPDeviceID

        vendor_match = re.search(r"VEN_([^&]+)", device_id)
        product_match = re.search(r"PROD_([^&]+)", device_id)
        serial_match = re.search(r"\\([^\\]+)$", device_id)

        vendor_id = vendor_match.group(1) if vendor_match else "UnknownVendor"
        product_id = product_match.group(1) if product_match else "UnknownProduct"
        serial_number = serial_match.group(1) if serial_match else "UnknownSerial"

        known = self.check_known(vendor_id, product_id, serial_number)

        # Predict anomaly if unknown
        anomaly_status = "N/A"
        if not known:
            input_df = pd.DataFrame([[vendor_id, product_id]], columns=["vendor_id",
"product_id"])
            input_encoded = pd.get_dummies(input_df)
            input_encoded = input_encoded.reindex(columns=self.columns, fill_value=0)
            if self.model:
                result = self.model.predict(input_encoded)[0]
                anomaly_status = "Anomalous" if result == -1 else "Normal"
            else:
                anomaly_status = "Unknown"

        # Log to DB
```

```
        self.log_to_db(timestamp, device_id, vendor_id, product_id, serial_number,
int(known), anomaly_status)

        # Console alert
        if anomaly_status == "Anomalous":
            print(f"[ALERT] Suspicious USB Detected at {timestamp}: Vendor {vendor_id},
Product {product_id}, Serial {serial_number}")
        else:
            print(f"[INFO] USB Inserted at {timestamp}: Vendor {vendor_id}, Product
{product_id}")

    def check_known(self, vendor_id, product_id, serial_number):
        conn = sqlite3.connect(DB_NAME)
        cursor = conn.cursor()
        cursor.execute('''
            SELECT COUNT(*) FROM usb_log
            WHERE vendor_id=? AND product_id=? AND serial_number=? AND known=1
        ''', (vendor_id, product_id, serial_number))
        result = cursor.fetchone()[0]
        conn.close()
        return result > 0

    def log_to_db(self, timestamp, device_id, vendor_id, product_id, serial_number,
known, anomaly_status):
        conn = sqlite3.connect(DB_NAME)
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO usb_log (timestamp, device_id, vendor_id, product_id,
serial_number, known, anomaly_status)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        ''', (timestamp, device_id, vendor_id, product_id, serial_number, known,
anomaly_status))
        conn.commit()
        conn.close()

# ------------------ HELPER TO TAG KNOWN DEVICES ------------------
def mark_known_usb(vendor_id, product_id, serial_number):
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()
    cursor.execute('''
        UPDATE usb_log SET known = 1
        WHERE vendor_id=? AND product_id=? AND serial_number=?
    ''', (vendor_id, product_id, serial_number))
    conn.commit()
    conn.close()
    print(f"[INFO] Marked USB as known: {vendor_id}, {product_id}, {serial_number}")

# ------------------ HELPER TO FIND MATCHING PHYSICAL DISK ------------------
def find_matching_disk(wmi_obj, drive_letter):
```

```
    max_retries = 5
    for _ in range(max_retries):
        for disk_drive in wmi_obj.InstancesOf("Win32_DiskDrive"):
            if "USB" in disk_drive.InterfaceType:
                # Get partitions associated with this disk
                partitions = wmi_obj.ExecQuery(
                    f"ASSOCIATORS OF
{{Win32_DiskDrive.DeviceID='{disk_drive.DeviceID}'}} "
                    f"WHERE AssocClass = Win32_DiskDriveToDiskPartition"
                )
                for partition in partitions:
                    # Get logical disks associated with this partition
                    logical_disks = wmi_obj.ExecQuery(
                        f"ASSOCIATORS OF
{{Win32_DiskPartition.DeviceID='{partition.DeviceID}'}} "
                        f"WHERE AssocClass = Win32_LogicalDiskToPartition"
                    )
                    for logical_disk in logical_disks:
                        if logical_disk.DeviceID == drive_letter:
                            return disk_drive
        time.sleep(1)
    return None

# ------------------- USB MONITORING -------------------
def monitor_usb():
    pythoncom.CoInitialize()
    handler = USBEventHandler()
    wmi = win32com.client.Dispatch("WbemScripting.SWbemLocator").ConnectServer(".",
"root\\cimv2")

    watcher = wmi.ExecNotificationQuery("SELECT * FROM Win32_VolumeChangeEvent WHERE
EventType = 2")
    print("🔌 USB Monitoring Started. Insert a USB storage device...")

    while True:
        try:
            usb_event = watcher.NextEvent()
            drive_letter = usb_event.DriveName

            disk = find_matching_disk(wmi, drive_letter)
            if disk:
                handler.on_usb_inserted(disk)
            else:
                print(f"[INFO] USB inserted at {drive_letter}, but no matching
DiskDrive info found.")

        except Exception as e:
            print("[ERROR]", e)
            time.sleep(1)
```

```python
# -------------------- MAIN --------------------
if __name__ == "__main__":
    init_db()
    try:
        monitor_thread = threading.Thread(target=monitor_usb)
        monitor_thread.start()
    except KeyboardInterrupt:
        print("\n[INFO] Monitoring stopped by user.")
```

```
USB Monitoring Started. Insert a USB storage device...
[INFO] USB Inserted at 2025-08-05 14:40:16: Vendor SANDISK, Product CRUZER_BLADE
```



INFERENCE

The USB monitoring system effectively detects and logs USB device insertions in real time, extracting key device details. It uses anomaly detection to identify suspicious or unauthorized devices and raises alerts accordingly, enhancing security. The system also maintains a device history for ongoing analysis and model training.